

Java and Maven installation

We need Java and Maven to build a package file for the sample application that we are going to run in a Docker container. Install Java and Maven in the Ubuntu OS. Once installation is completed successfully, verify the Java version using the `java -version` command.

```
osfy@ubuntu-22-04-1-lts:~/Desktop$ java -version
openjdk version "11.0.17" 2022-10-18
OpenJDK Runtime Environment (build 11.0.17+8-post-Ubuntu-1ubuntu22.04)
OpenJDK 64-Bit Server VM (build 11.0.17+8-post-Ubuntu-1ubuntu22.04, mixed mode, sharing)
```

Verify the Maven version using the `mvn -version` command:

```
osfy@ubuntu-22-04-1-lts:~/Desktop$ mvn -version
Apache Maven 3.6.3
Maven home: /usr/share/maven
Java version: 11.0.17, vendor: Ubuntu, runtime: /usr/lib/jvm/
java-11-openjdk-amd64
Default locale: en_IN, platform encoding: UTF-8
OS name: "linux", version: "5.15.0-53-generic", arch:
"amd64", family: "unix"
```

We will use PetClinic as a sample application. PetClinic is a Spring Boot application built using Maven or Gradle. In this article we will build a JAR file and deploy it in a minikube cluster. Visit <https://github.com/spring-projects/spring-petclinic> to know more. Copy the GitHub repository URL from the above URL and use the `git clone` command to clone it locally:

```
osfy@ubuntu-22-04-1-lts:~/Desktop$ git clone https://github.
com/spring-projects/spring-petclinic.git
```

Once the source code is available, go to the root of the project using the `change directory` command as given below, and execute the `./mvnw package` command to create a JAR file. At first execution it will try to download multiple dependencies and hence will take some time to complete execution.

```
osfy@ubuntu-22-04-1-lts:~/Desktop$ cd spring-petclinic
osfy@ubuntu-22-04-1-lts:~/Desktop/spring-petclinic$ ./mvnw
package
```

```
Warning: JAVA_HOME environment variable is not set.
[INFO] Scanning for projects...
Downloading from spring-snapshots: https://repo.spring.
io/snapshot/org/springframework/boot/spring-boot-starter-
parent/2.7.3/spring-boot-starter-parent-2.7.3.pom
```

```
.
.
Downloaded from central: https://repo.maven.apache.
org/maven2/org/springframework/session/spring-session-
bom/2021.2.0/spring-session-bom-2021.2.0.pom (3.1 kB at 6.0
kB/s)
[INFO]
[INFO] -----< org.springframework.samples:spring-
petclinic >-----
[INFO] Building petclinic 2.7.3
[INFO] -----[ jar ]-----
.
.
[INFO] Building jar: /home/osfy/Desktop/spring-petclinic/
target/spring-petclinic-2.7.3.jar
[INFO]
[INFO] --- spring-boot-maven-plugin:2.7.3:repackage
(repackage) @ spring-petclinic ---
[INFO] Replacing main artifact with repackaged archive
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15:36 min
[INFO] Finished at: 2022-12-01T00:56:56+05:30
[INFO] -----
```

Observe the output of Maven command execution where it says *Building jar: /home/osfy/Desktop/spring-petclinic/target/spring-petclinic-2.7.3.jar*.

Now it is confirmed that we have a JAR file ready. The next step is to create a Docker image that contains the JAR file and runtime environment.

So, what is a Dockerfile? Well, it's nothing but the text document that provides build instructions to build the image with your application package.

```
FROM openjdk:11
MAINTAINER osfy
COPY target/spring-petclinic-2.7.3.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Place the Dockerfile in the *project root* directory. Now, it is time to build a Docker image.

For testing locally with minikube, we should point the local Docker daemon to the minikube internal Docker registry, and build the image to install it in the minikube cluster: `eval $(minikube docker-env)`.

Building a Docker image and running a container

Execute the `sudo docker image build -t petclinic:latest` command from the *project root* directory.

```
osfy@ubuntu-22-04-1-lts:~/Desktop$ cd spring-petclinic/
osfy@ubuntu-22-04-1-lts:~/Desktop/spring-petclinic$ ls
```